

Problem Analysis and Requirements Definition

=====

1. Problem Analysis

Purpose:

The Student Mark Estimator System is designed to help educators and administrators identify and fill missing assessment marks in student datasets. The application must make it easy to load a CSV grade file, detect missing values, estimate those values using a data-driven model, and export a corrected version of the dataset.

Current Situation:

- Teachers or administrators often maintain student assessment records in spreadsheet files.
- Some student grades may be missing due to absent submissions, incomplete grading, or data entry errors.
- Missing values make overall analysis, reporting, and grade calculations incomplete or unreliable.
- Manual review and estimation of missing marks is time-consuming and inconsistent.

Problem Statement:

Educators need a reliable, user-friendly application that can load existing grade CSV files, identify missing marks, estimate appropriate values using historical data patterns, and export an updated dataset without overwriting the original file.

Requirements Statement:

The system shall allow educators to load a student grade CSV, identify missing assessment values, estimate those values using a transparent predictive process, and export the corrected dataset to a new file while preserving the original data. The system shall provide visual feedback, prevent invalid operations, and support both single-item and batch estimation.

Stakeholders:

- Educators and teachers who track student performance.
- School administrators who need accurate grade records.
- Students indirectly affected by corrected grade estimations.
- Developers maintaining the system.

Goals:

- Automatically detect missing grades in imported CSV datasets.
- Estimate missing marks using a transparent machine learning approach.
- Keep the interface simple and accessible for users with minimal technical experience.
- Export corrected datasets safely using a prefixed filename.
- Provide clear logs and feedback for each operation.

Scope:

In scope:

- Loading CSV files via file dialog or drag-and-drop.
- Detecting missing values in assessment columns.
- Estimating missing marks using linear regression trained on complete student data.
- Supporting single-item estimation and batch "run all" estimation.
- Exporting results as a new CSV file.
- Opening the exported file in the users default spreadsheet or text editor.

Out of scope:

- Editing individual cell values manually within the app.
- Complex ensemble or neural network models.
- Authentication, multi-user collaboration, or database integrations.
- Real-time grade synchronization with external systems.

Constraints:

- Input data must be in CSV format.
- The first column is treated as the student identifier.
- At least one assessment column beyond the ID column is required.
- Linear regression requires a minimum amount of complete data to train reliably.
- The application is built using Python and Tkinter, so it must remain within desktop GUI constraints.

Assumptions:

- Input CSV files contain a header row.
- Missing grades are represented by empty values or NaNs.
- Assessment columns are numeric and bounded between 0 and 100.
- Students are represented by unique identifiers in the first column.

2. Requirements Definition

Functional Requirements:

FR1: Load Dataset

- The application should allow the user to load a CSV file from disk using a browse dialog.
- The application should allow the user to load a CSV file by dragging and dropping it into the window.
- The first column of the loaded CSV should be interpreted as the student ID column.
- All remaining columns should be treated as assessment score columns.

FR2: Missing Data Detection

- The application should scan the loaded dataset for missing values in assessment columns.
- The application should display a list of each missing assessment item, including student ID and assessment column.
- If no missing values exist, the application should disable estimation controls and notify the user.

FR3: Estimation

- The application should allow the user to select a missing item and run a single estimation.
- The application should allow the user to estimate all missing items in one batch operation.
- The application should use linear regression to estimate missing marks when sufficient data exists.
- The application should clamp the estimated result between 0 and 100 and round to one decimal place.
- The application should show supporting metrics and context for each prediction.

FR4: Export Results

- The application should export the corrected dataset to a new CSV file with the prefix `estimated\_` added to the original filename.
- The application should not overwrite the original input CSV file.
- The application should allow the user to open the exported file in the default spreadsheet or text editor.

FR5: User Interface and Feedback

- The application should display a status label showing the loaded filename or a message when no file is loaded.
- The application should present log output describing success, errors, and estimation results.
- The application should provide a theme toggle for dark and light display modes.
- The application should disable buttons that are not valid until required input is available.

Non-functional Requirements:

NFR1: Usability

- The interface should be intuitive and suitable for users with limited technical experience.
- Error messages should be clear and actionable.

NFR2: Reliability

- The application should handle invalid or malformed CSV files gracefully and provide an error dialog.
- Export operations should be protected against file write failures.

NFR3: Performance

- The application should load and scan datasets quickly for typical classroom-sized files.
- Batch estimation should complete in a reasonable time for moderate datasets.

NFR4: Maintainability

- The implementation should remain modular so that data loading, scanning, prediction, and export logic can be updated independently.
- Comments and logs should support future debugging and enhancement.

NFR5: Portability

- The application should run on Windows desktops where Python, Tkinter, and required libraries are installed.

3. Success Criteria

- A valid CSV file can be loaded and parsed without crashing.
- Missing assessment entries are identified and listed correctly.
- A selected missing mark can be estimated and exported successfully.
- A full batch estimation can run and save a new CSV file.
- The user can open the exported file from within the app.
- Theme toggling updates the interface style consistently.

4. Risks and Mitigations

- Risk: Insufficient complete training data for linear regression.
Mitigation: Provide fallback messaging and require more complete rows before estimation.
- Risk: Non-CSV files or malformed CSV format.
Mitigation: Validate file extension and catch parsing exceptions with user alerts.
- Risk: Export file write permissions or path errors.
Mitigation: Wrap export logic in error handling and notify the user when export fails.

Algorithm Pseudocode (Linear Regression)

```
-----  
FUNCTION PredictMissingMark(student_data, historical_dataset):  
    X = historical_dataset.features (Midterm, Assignments, Attendance)  
    y = historical_dataset.target (Final Mark)  
  
    IF length(historical_dataset) < 3:  
        RETURN FallbackToClassAverage(historical_dataset)  
  
    model = FitLinearRegression(X, y)  
    prediction = model.predict(student_data.features)  
  
    # Post-processing & Boundary Checks  
    prediction = Clamp(prediction, 0, 100)  
    prediction = RoundTo(prediction, 1)  
  
    RETURN prediction  
END FUNCTION
```

=====

DESIGN JUSTIFICATION

Linear Regression was selected because it is transparent, interpretable, and well-suited to small datasets of numeric assessment scores. In this application, educators need to understand and trust how missing marks are estimated, so a model with clear behavior was preferred. Linear Regression also performs efficiently on low-dimensional data and integrates directly with the Python scientific stack already used in the project.

This design avoids more complex machine learning approaches because the dataset is typically small, the goal is prediction of numeric scores, and users require easy explanation of the results. The application also includes fallback logic to ensure stable output when the data quality is insufficient for reliable regression training.

DATA PRE-PROCESSING

-
- Input validation checks that the CSV is not empty and contains at least two columns.
 - The first column is assumed to be the student identifier; all other columns are treated as numeric assessment scores.
 - Missing values are detected with `DataFrame.isna()` on assessment columns.
 - Training data is assembled by dropping rows where the target or required feature columns are missing, using `df.dropna(subset=[target_col] + feature_cols)`.
 - No explicit feature scaling is applied because assessment scores are assumed to share a

common 0-100 grading scale.

- When there is insufficient complete training data, the application uses mean-based fallbacks instead of attempting to train an unreliable model.
- The export step writes the updated DataFrame to a new CSV file prefixed with `estimated\_`.

MODEL SELECTION

The selected model is `sklearn.linear\_model.LinearRegression`.

Justification:

- The task is numeric prediction of missing assessment scores from other numeric scores.
- Linear regression is a well-understood model that is easy to explain to educators.
- The model performs well on small-to-medium datasets and requires minimal configuration.
- It is computationally efficient, which is important for a desktop application.
- Alternative models such as decision trees, k-nearest neighbors, or neural networks were not chosen because they introduce additional complexity and are less transparent for users.
- The app also includes fallback logic for cases with too few complete rows, which preserves reliability when the dataset does not support regression training.

SYSTEM STRUCTURE

1. Application Architecture

- The system is implemented as a single desktop application using Python and Tkinter, with a single top-level controller class: `MarkEstimatorApp`.
- `MarkEstimatorApp` manages UI construction, state management, data loading, missing-value detection, prediction, results export, and theming.
- The program uses a Model-View-Controller-like separation internally: UI components are created and updated separately from data processing and prediction logic.

2. Components

- UI Layer
 - * File selection controls and status label.
 - * Missing-data list box for presenting targets.
 - * Action buttons for single estimation, batch estimation, and file opening.
 - * Log output text area for status, progress, and prediction details.
- Data Layer
 - * `self.df`: loaded `pandas.DataFrame` representing the CSV dataset.
 - * `self.id\_col`: header of the first column, treated as the student ID.
 - * `self.assessment\_cols`: list of all assessment score columns.

- * ``self.missing_records``: list of dictionaries representing every missing score target.
- Processing Layer
 - * CSV loading and validation.
 - * Missing value scanning and target generation.
 - * Prediction engine with fallback strategies.
 - * Export functionality.

3. User Interface Layout

- Main window contains three primary sections:
 1. Load Dataset
 - "Browse CSV File" button.
 - File status label showing loaded file name or "No file loaded."
 2. Select Missing Mark to Estimate
 - A scrollable ``Listbox`` showing missing entries in the format: "Student ID: <id> | Missing Task: <column>".
 3. Actions and Results
 - Buttons: "Run ML Estimation", "Run All Estimations", and "Open in Spreadsheet / Text Editor".
 - A log area displaying messages, results, and progress.
- The app supports drag-and-drop for CSV files via ``tkinterdnd2``.
- A theme toggle switches between dark mode and light mode, updating colors and widget styles dynamically.

4. Data Flow

- Load CSV
 - * User browses or drops a CSV file.
 - * Application validates file existence, non-empty content, and at least two columns.
 - * First column becomes ``id_col``; remaining columns become ``assessment_cols``.
- Scan for Missing Data
 - * The app identifies rows where any assessment column is missing.
 - * Every missing cell is added to ``self.missing_records`` and displayed.
 - * If no missing values are found, estimation buttons are disabled.
- Estimate Missing Value
 - * User selects one target and clicks "Run ML Estimation".
 - * ``predict_mark()`` is invoked and returns a value plus metrics.
 - * Result is logged and written to an exported file.
- Batch Estimation
 - * User clicks "Run All Estimations" to process every missing target.
 - * Each missing entry is estimated and written into the in-memory DataFrame.
 - * A single output CSV is saved after processing all targets.
- Export / Open

- * The corrected dataset is saved with prefix `estimated\_`.
- * The exported file can be opened using the OS default editor.

5. Prediction Algorithm

- Input
 - * Target student ID and target assessment column.
 - * Feature columns are all other assessment columns.
- Fallback Logic
 - * If there is only one assessment column, use the target column mean.
 - * If there are fewer than two complete rows to train on, use the target column mean.
 - * If the target student is missing scores for other feature columns, use the student's own average across available feature scores.
- Linear Regression Path
 - * Train on rows with complete values for both the target and feature cols.
 - * Use `sklearn.linear\_model.LinearRegression`.
 - * Predict the missing score for the target student.
 - * Clamp predicted value to `[0.0, 100.0]` and round to one decimal place.
- Metrics
 - * `Class Baseline Mean for <target\_col>`.
 - * `Target Student Mean on other tests`.
 - * `Robust Sample Sizes Used for Training`.

6. Error Handling and Validation

- File validation rejects empty CSV files and datasets with fewer than two columns.
- Drag-and-drop rejects non-CSV file extensions.
- File parsing exceptions are caught and reported using modal error dialogs.
- Selection warnings prevent running estimation without a highlighted item.
- Export operations handle write failures and notify the user.
- Opening files uses OS-specific launch commands for Windows, macOS, and Linux.

7. External Dependencies

- `tkinter` and `tkinter.ttk` for GUI controls.
- `tkinterdnd2` for drag-and-drop file loading.
- `pandas` for CSV reading, DataFrame manipulation, and exporting.
- `scikit-learn` for `LinearRegression` prediction.
- `os`, `sys`, and `subprocess` for file path handling and editor launch.

8. Design Decisions

- Use a desktop GUI because the application is intended for local grade file maintenance.
- Keep the code in a single class for simplicity, while still separating

UI construction from prediction logic.

- Export with `estimated\_` prefix to preserve the original dataset.
- Use linear regression for explainability, with clear fallback methods when training data is insufficient.
- Provide both single-item and batch estimation modes for flexibility.

9. Future Enhancements

- Add a preview table of the loaded dataset before estimation.
- Allow manual adjustment of estimated values before export.
- Support custom target column selection instead of assuming the first column is always student ID.
- Add data normalization or scaling options for improved regression quality.
- Save user preferences for theme mode and last opened folder.

Development Documentation

=====

1. Implementation Overview

The Student Mark Estimator System is implemented as a Python desktop application using Tkinter.

The main program is contained in ``CODE/mark_estimator_experimental.py`` and is centered on a single controller class, ``MarkEstimatorApp``.

Key implementation elements:

- ``tkinter`` and ``ttk`` for GUI widgets and layout.
- ``tkinterdnd2`` for drag-and-drop CSV file loading.
- ``pandas`` for CSV parsing, DataFrame management, missing value handling, and export.
- ``scikit-learn`` ``LinearRegression`` for predicting missing marks.
- ``subprocess`` and ``sys`` to open exported files with the OS default editor.

2. Development Tasks and Workflow

- Set up the main application class and root window initialization.
- Build the UI components in ``_create_widgets()``:
 - * File load controls and status label.
 - * Missing data list box with scrollbar.
 - * Buttons for estimation, batch processing, and opening exported files.
 - * Log output text area.
- Implement theme switching and style updates in ``_update_theme_styles()``.
- Implement CSV loading with validation in ``_load_csv_from_path()``.
- Implement missing-data scanning in ``scan_for_missing_data()``.
- Implement single missing item estimation with ``process_estimation()``.
- Implement batch estimation with ``process_all_estimations()``.
- Implement prediction logic with fallbacks in ``predict_mark()``.
- Implement export logic in ``export_results()``.
- Implement OS-aware file opening in ``open_exported_file()``.
- Add user messaging and error handling for robustness.

3. Source File Structure

- ``mark_estimator_experimental.py``
 - * import section
 - * ``MarkEstimatorApp`` class
 - * application initialization under ``if __name__ == "__main__":``

Inside `MarkEstimatorApp`:

- `__init__()` initializes app state, theme mode, styles, and drag/drop support.
- `_update_theme_styles()` applies colors, widget styles, and theme-specific behavior.
- `_create_widgets()` constructs the GUI layout and widgets.
- `log_message()` appends messages to the output log.
- `load_csv()`, `handle_drop()`, `_load_csv_from_path()` manage file loading.
- `scan_for_missing_data()` finds and lists missing assessment cells.
- `process_estimation()` handles the selected missing score estimation path.
- `process_all_estimations()` handles batch estimation for all missing scores.
- `predict_mark()` chooses a prediction method and returns estimate metrics.
- `export_results()` writes the corrected dataset to a new CSV file.
- `open_exported_file()` launches the exported file in a system editor.

4. Key Implementation Notes

- The first CSV column is assumed to be the student identifier.
- All subsequent columns are treated as assessment scores.
- The application uses `pd.read_csv()` for loading datasets.
- Empty or malformed CSV files are rejected with user-facing error dialogs.
- Missing data is detected by checking `DataFrame.isna().any(axis=1)` on assessment columns.
- When training data is insufficient, prediction falls back to simple averages.
- Estimates are clamped between 0 and 100 and rounded to one decimal place.
- The exported file is prefixed with `estimated_` to preserve the original input.
- `self.last_exported_file` stores the last generated file path.
- The "Open in Spreadsheet / Text Editor" button is enabled only after successful export.

5. Testing and Validation

Manual testing should include:

- Loading a valid CSV file with complete and missing values.
- Drag-and-drop loading of a CSV file.
- Loading an empty CSV and verifying the error dialog.
- Attempting to load a non-CSV file via drag-and-drop and verifying rejection.
- Selecting a missing item and running single estimation.
- Running batch estimation and checking exported file contents.
- Opening the exported file from the app.
- Switching between dark and light themes and verifying UI updates.
- Verifying that the original CSV is not modified.

6. Known Limitations

-
- The app assumes the first column is always the student identifier.
 - Prediction is based on linear regression only and may not be optimal for all datasets.
 - There is no manual edit mode for individual data cells inside the UI.
 - The application does not persist user preferences across sessions.
 - The UI does not currently show a full data table preview.

7. Future Development Ideas

- Add a dataset preview table for the loaded CSV.
- Allow the user to choose which column is the student identifier.
- Add manual editing of individual predicted values before export.
- Add more advanced regression options or model selection.
- Include a configuration file for default export location and theme.
- Implement automated testing for data loading and prediction behavior.
- Add progress indicators for large batch estimations.

Evaluation Report

=====

1. Effectiveness

- The application successfully loads CSV datasets and identifies missing assessment values.
- The missing-data list clearly presents missing entries by student ID and assessment column.
- The single-item estimation flow uses regression and fallback logic to estimate missing marks and returns a usable predicted score.
- The batch estimation feature processes multiple missing entries in one pass and exports a complete corrected dataset.
- Exported files are saved with the prefix `estimated\_`, preserving the original dataset.
- The UI provides good user feedback through log messages and modal dialogs.
- Theme switching works consistently and updates both widget styles and log area colors.

2. Limitations

- The application assumes the first CSV column is a unique student identifier; it does not support explicit ID column selection.
- The prediction model is limited to linear regression and simple average fallbacks, which may not be optimal for complex grading patterns.
- If the target student is missing values in multiple feature columns, the fallback to student mean may produce lower-quality estimates.
- No manual correction interface exists for users to adjust estimates before export.
- The current UI does not display a full dataset preview for review before estimation.
- There is no persistence of user preferences (such as theme mode or last opened folder) between sessions.
- Error handling is present but could be expanded to detect additional data quality issues like non-numeric score entries or inconsistent row lengths.

3. Improvements

- Add an explicit column selection step so users can choose the student identifier and target score columns.
- Add a dataset preview table to allow users to inspect loaded data before running estimates.
- Introduce more advanced model options, such as ridge regression, k-nearest neighbors, or decision trees, with a model selection control.
- Implement manual review and override of suggested estimated values before saving.
- Add validation for numeric-only assessment columns and provide clearer reports for malformed data.
- Persist user settings such as theme preference and last opened folder path.
- Add unit tests and integration tests for data loading, prediction fallbacks, export behavior,

and error conditions.

- Improve export handling by supporting automatic folder selection and naming conflict resolution.
- Enhance logging with timestamped entries and an optional exportable operation report.

4. Social and Ethical Considerations

- Predicted grades should be treated as estimates and not automatic final scores. Human review is required before any estimate is accepted into official records.
- The model is trained only on available assessment data, which may reflect existing grading patterns or incomplete sampling. This creates a risk of reinforcing bias present in the data.
- Missing data is often not random: absent or underperforming students may be overrepresented in missing records, and blind prediction could unfairly alter their outcomes.
- The application avoids using demographic or personal information as features, relying instead only on assessment scores to reduce privacy and fairness risks.
- Mitigation strategies include preserving the original dataset, making the export file clearly separate, documenting model limitations, and encouraging manual validation of any estimates.
- Users should be cautioned that model-based estimates are a support tool and not a substitute for professional academic judgment.

5. Overall Assessment

The Student Mark Estimator System is functional for its core purpose of identifying missing assessment marks and populating them with estimated values. It is effective for typical classroom datasets, offers a usable interface, and provides both single and batch estimation modes. However, the solution is best suited for small to medium datasets and would benefit from stronger validation, more flexible column handling, and richer prediction options for production use.